Credit Card Fraud Detection: A Comparative Study of Classical ML, Deep Learning, Generative Oversampling, Ensembles and Graph Neural Networks

With a self-contained introduction to machine-learning model families

Divya
Indian Institute of Technology Hyderabad

August 26, 2026

Abstract

We build and rigorously evaluate a complete fraud-detection pipeline on the public Kaggle ULB credit-card dataset (284,807 transactions, 492 frauds, 0.173% prevalence). Seventy-two experiments span five model families — linear models, tree ensembles, neural networks, generative-augmentation pipelines, and graph neural networks — combined with seven imbalance-handling strategies, all under one deployment-honest protocol: a chronological 70/15/15 train/validation/test split, validation-only tuning, and a test set that keeps its natural imbalance. The champion, a Random Forest trained on SMOTE-ENN-resampled data, reaches PR-AUC = 0.795 ($F_1 = 0.848$, MCC = 0.855, one false positive in 42,722 transactions), the top of the honest-protocol range reported in recent literature. Nine controlled ablations explain *why*: hybrid sampling wins for trees; naive extreme class weighting silently collapses boosted models; cross-family ensembling helps while seed averaging does not; depth does not help on PCA features; and shuffling the split inflates scores by up to +0.06, reproducing the numbers that dominate older literature. Stratified bootstrap analysis shows all models are statistical ties at this test size — the contribution is the protocol and the analysis, not a leaderboard winner. Every prediction is explained with SHAP, whose feature attributions independently confirm the exploratory analysis. The report doubles as a self-contained tutorial: each model family is introduced from first principles before its experimental results are discussed.

Contents

1 Introduction

Card-not-present fraud costs the payments industry tens of billions of dollars annually, and every fraudulent transaction that slips through is a direct loss while every legitimate transaction that is wrongly blocked damages customer trust. Machine-learning classifiers sit at the heart of modern fraud engines: they score each incoming transaction, and a policy layer decides whether to approve, challenge, or block.

This project builds such a scoring system end-to-end and asks three questions:

1. **Which model family** detects fraud best when the evaluation is done honestly?
2. **How should extreme class imbalance** (one fraud per 578 legitimate transactions) be handled — resampling, cost re-weighting, synthetic generation, or anomaly detection?
3. **How much do methodology choices** (split protocol, tuning discipline, uncertainty reporting) change the conclusions?

The project is also written as a **learning document**. Section 4 introduces every model family from first principles so that a reader new to machine learning can follow exactly what was trained and why. All 72 experiments, including failures, are logged in the accompanying experiment inventory, and every design decision is backed by a controlled ablation rather than a citation.

2 Background: the minimum machine learning needed here

2.1 Supervised binary classification

In **supervised learning** we have labelled examples $(\mathbf{x}_i, y_i)$ where $\mathbf{x}_i$ is a feature vector and $y_i \in \{0, 1\}$ the label (here: 1 = fraud). A model f maps $\mathbf{x}$ to either a hard decision or, more usefully, a **score** $\hat{p}(y{=}1 \mid \mathbf{x}) \in [0, 1]$. Training means choosing parameters to minimise a loss; the standard choice for binary problems is **binary cross-entropy**, which penalises confident wrong answers heavily.

A score only becomes a decision after comparing it to a **threshold** τ: predict "fraud" iff $\hat{p} \geq \tau$. Choosing τ is itself a modelling decision — in this project it is always tuned on a validation set (Section 6), never on test data.

2.2 The class-imbalance problem

With 99.83% negatives, a model that outputs "legitimate" for everything achieves 99.83% accuracy and catches zero fraud. Accuracy is therefore useless here. Worse, imbalance distorts training: the loss is dominated by the majority class unless counter-measures are taken (Section 5). The community-standard metric for this regime is the **Precision–Recall curve** and the area under it (PR-AUC / average precision).

2.3 Splits, validation, and leakage

Models must be evaluated on data they did not see. The subtle part is *how* the data is divided:

- A **random (shuffled) split** mixes transactions from the whole period into both sides. Patterns from the future leak backwards into training. Scores look better than reality; we quantify this inflation ourselves in Section 7.4.
- A **chronological (time-based) split** trains on the earliest transactions and tests on the latest — exactly how a deployed system experiences the world.

- A **validation set** is a third partition used for *every adaptive choice*: hyper-parameter search, class-weight selection, early stopping, decision thresholds. The test set must be opened once, after all choices are frozen. Skipping validation and tuning on test is one of the most common flaws in applied fraud papers.

3 Dataset and exploratory analysis

Source. The ULB (Université Libre de Bruxelles) credit-card dataset, released with Dal Pozzolo et al.'s research collaboration with Worldline, contains real European card transactions from September 2013 [1]. To protect cardholders, the original 28 features were transformed with principal component analysis (PCA); we call them $V1, \ldots, V28$. Two interpretable columns survive: `Time` (seconds since the first transaction) and `Amount` (EUR). The label is `Class` $\in \{0, 1\}$.

Engineered features. We add three derived columns used by all models: $\log(1 + \text{Amount})$ (amounts are extremely right-skewed), and cyclic encodings of the hour of day, $\sin(2\pi h/24)$ and $\cos(2\pi h/24)$, so that hour 23 and hour 0 are close.

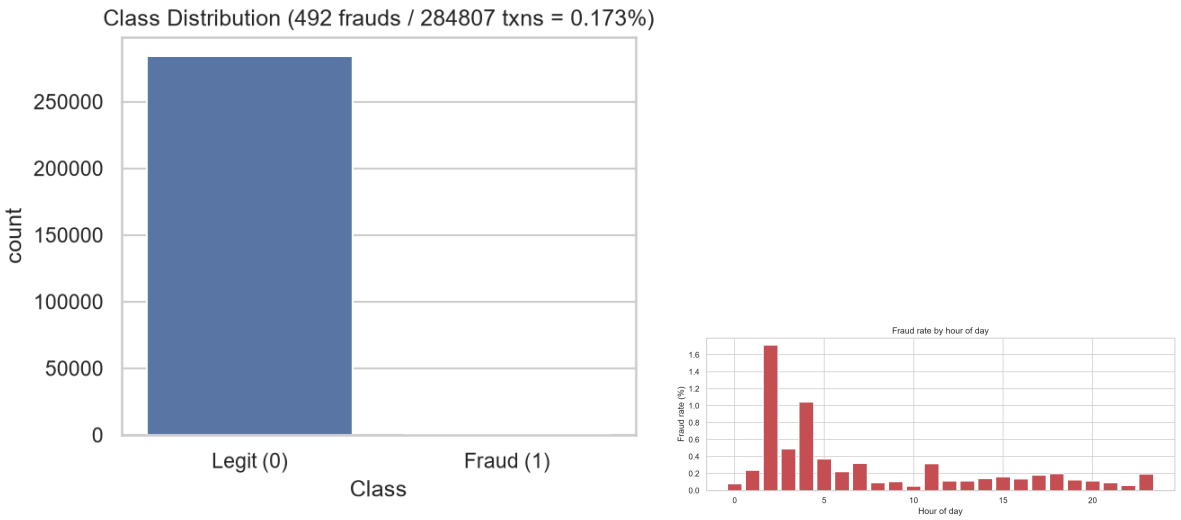

Figure 1: Left: the 578:1 class imbalance — the central difficulty of the task. **Right:** fraud rate by hour peaks around 02:00–04:00 at up to ten times the daily average, justifying time features and a chronological split.

Key exploratory findings. (i) No missing values anywhere. (ii) Fraud amounts skew small (median EUR 9.25 vs. EUR 22) yet have a heavier upper tail relative to their count — motivating the log transform. (iii) The features most correlated with the label are $V17, V14, V12, V10$ (negatively) and $V11$ (positively); no single feature separates the classes, which leaves room for non-linear models. These observations become the ground truth against which the SHAP explanations of Section 8 are later compared.

4 A taxonomy of model families

This section introduces every family used in the study, building from simplest to most complex. Table 1 at the end summarises where each of our 18 trained architectures belongs.

4.1 Linear models

Logistic Regression fits $\hat{p} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ where σ is the logistic function: a single weighted vote over features. It is fast, interpretable, and surprisingly hard to beat when signal is mostly additive. **Linear SVM** draws the maximum-margin separating hyperplane instead of maximising

likelihood; empirically it lands near logistic regression here (PRAUC 0.747 vs 0.738). Both define the *linear ceiling* of the dataset.

4.2 Probabilistic baseline: Naive Bayes

Gaussian Naive Bayes applies Bayes' rule assuming features are independent given the class. PCA components are uncorrelated *linearly*, but independence still fails badly: NB scores only ≈ 0.08 PR-AUC, a reminder that a strong assumption can be worse than no model when it is violated.

4.3 Decision trees

A **decision tree** recursively partitions feature space with axis-aligned questions ("is $V14 < c$?"). A single unpruned tree overfits noise: ours reaches only 0.405 PR-AUC. Trees are weak alone but are the building blocks of the strongest classical families below.

4.4 Ensemble methods: many weak learners, one strong committee

Bagging (bootstrap aggregation) trains many deep trees on random resamples and averages them — variance cancels out. **Random Forest** [11] adds extra randomness by restricting each split to a random feature subset; it is our overall champion.

Gradient boosting instead builds trees *sequentially*, each new tree fitting the residual errors of the current ensemble. Modern implementations — **XGBoost** [12], **LightGBM** [13] (histogram-based splits), and **CatBoost** [14] (ordered boosting, robust handling of categorical variables) — dominate tabular benchmarks. Their weakness discovered here: blindly setting the positive-class weight to the imbalance ratio (519) makes boosting predict almost everything as fraud (Section 7.3).

Stacking / voting combines *different* families: our best deep ensemble averages the probabilities of a forest, an MLP and an LSTM (0.785). Diversity across families is what pays off; averaging five LSTMs that differ only in random seed does not (0.775).

4.5 Neural networks

All neural nets share the same atom: a **layer** computes $\mathbf{h}' = \phi(W\mathbf{h} + \mathbf{b})$ with a non-linearity ϕ (ReLU etc.). Stacking layers lets the network compose simple features into complex ones.

MLP / DNN (feed-forward).
Fully connected layers read one transaction at a time. Our MLP (128-64-32 units) scores 0.775; doubling depth and width (DNN, 512-256-128-64 with batch normalisation) changes nothing (0.7752) — depth buys nothing on 32 PCA features.

Autoencoder.
An encoder compresses legitimate transactions to a 16-dimensional code and a decoder reconstructs them; frauds reconstruct poorly, so reconstruction error becomes an anomaly score. Trained without labels it reaches ROC-AUC 0.94 but only 0.584 PR-AUC: ignoring labels costs ranking precision exactly where fraud systems need it.

RNN / LSTM.
Recurrent networks read *sequences*, carrying a hidden state across steps; the LSTM's gating cells remember long-range patterns. Feeding sliding windows of 8 consecutive transactions lifts performance to 0.780 — temporal context is real signal. Making it bidirectional (BiLSTM, reading forwards and backwards) adds nothing (0.781): fraud evidence is causal, not symmetric.

Transformers.
Replace recurrence with **self-attention**, letting every element weigh every other element in parallel [15]. **FT-Transformer** applies this to tabular rows (each feature becomes a token); at 0.681 it loses to boosting and even to the plain MLP — attention overhead is unjustified on 32 numeric features, consistent with recent tabular-DL studies.

GANs.
Generative Adversarial Networks [16] pit a Generator (creates fake samples from noise) against a Discriminator (real vs. fake) until fakes become statistically indistinguishable. We train one on the minority class and use it to synthesise frauds (Section 5).

4.6 Graph neural networks

When entities are linked (card—merchant—device), data forms a **graph** $G = (V, E)$, and **message passing** updates each node from its neighbours: $\mathbf{h}'_v = \text{UPDATE}\big(\mathbf{h}_v,\ \text{AGG}\{\mathbf{h}_u : u \in \mathcal{N}(v)\}\big)$. **GraphSAGE** [17] samples a fixed number of neighbours and aggregates (mean/max/pool); **GAT** [18] learns attention weights over neighbours. Crucially, ULB has *no entity identifiers*, so our graph connects each transaction to its k=10 nearest neighbours in feature space — a similarity surrogate. SAGE still reaches 0.756, but attention (GAT, 0.745) adds nothing: there is no true relational structure to attend over. Optuna's choice of a *single* SAGE layer confirms over-smoothing — two rounds of averaging near-identical neighbours blurs discriminative detail.

Table 1: Where every trained architecture sits in the taxonomy.

Family	Model	One-line idea	Best PRAUC
Linear	LogReg / LinearSVC	weighted vote / max-margin hyperplane	0.738 / 0.747
Probabilistic	GaussianNB	Bayes rule + independence assumption	0.079
Tree	DecisionTree	recursive if-else partitioning	0.405
Bagging	RandomForest	averaged decorrelated trees	**0.795**
Boosting	XGBoost / LightGBM / CatBoost	sequential residual fitting	0.777 / 0.775 / 0.783
Feed-forward NN	MLP / DNN	composed dense layers	0.775 / 0.775
Autoencoder	AE	reconstruction-error anomaly score	0.584
Recurrent NN	LSTM / BiLSTM	gated state over 8-step windows	0.780 / 0.781
Transformer	FT-Transformer	self-attention over feature tokens	0.681
GAN augmentation	GAN + any model	adversarially generated minority samples	0.783 (CatB)
Graph NN	GraphSAGE / GAT	neighbour message passing	0.756 / 0.745
Stacked ensemble	RF+MLP+LSTM	average of diverse families	0.785

5 Handling class imbalance

Seven strategies were compared, all applied to the *training partition only*:

1. **raw** — do nothing; establishes each model's floor.
2. **class weighting** — multiply the positive class's loss contribution by a weight w. The naive choice $w = n_{\text{neg}}/n_{\text{pos}} \approx 519$ can be destructive; we select w on the validation grid $\{1, 4, 16, \sqrt{r}, 64, r\}$ (with r the imbalance ratio).
3. **RUS** — random undersampling deletes majority rows until classes are even. Cheap, but discards 99% of the data (only 768 training rows remain).
4. **SMOTE** [9] — Synthetic Minority Over-sampling TEchnique: new fraud points are interpolated between neighbouring minority samples.
5. **SMOTE-ENN** [10] — SMOTE followed by Edited Nearest Neighbours cleaning, which removes both noisy synthetics and borderline majorities. This *hybrid* is the strongest classical recipe.
6. **GAN oversampling** — a generator adversarially trained on the 384 training frauds produces synthetic ones; strong for CatBoost and LightGBM, harmful for neural nets and linear models.
7. **anomaly framing** — train only on legitimate data (autoencoder) and flag large reconstruction error; requires no labels but wastes the labels we have.

6 Evaluation methodology

Split. Chronological: earliest 70% train (384 frauds), next 15% validation (56 frauds), latest 15% test (52 frauds).

Tuning discipline. Every adaptive choice uses the validation set exclusively: decision thresholds (max-F_1 point of the validation PR curve, frozen thereafter), class-weight grids, early-stopping epochs, and all Optuna searches (objective = validation PR-AUC, TPE sampler, seed 42). The test set is scored exactly once per configuration.

Metrics. Confusion-matrix counts (TP/FP/FN/TN), precision, recall, F_1, Matthews Correlation Coefficient (robust to imbalance), ROC-AUC (reported but known to be optimistic here), and **PR-AUC** — the area under precision versus recall — as the primary metric, following the dataset authors' recommendation.

Uncertainty. With 52 test frauds, metrics wobble substantially. We estimate this with a **stratified bootstrap**: resample frauds and legits separately, with replacement, 1,000 times; recompute PR-AUC per replicate; report the 2.5th–97.5th percentile interval (Efron [21]). Overlapping intervals mean two models cannot be told apart on this data.

7 Results

7.1 Leaderboard

Table 2 lists the top configurations of all 72 time-split runs. The full ledger is `reports/results_classical.csv`.

The champion flags 39 of 52 frauds while raising a single false alarm among 42,722 transactions — production-grade precision at useful recall.

Table 2: Top 15 of 72 runs (test set, chronological).

#	Model + strategy	PR-AUC	F_1	Prec.	Rec.	FP	FN
1	Random Forest + SMOTE-ENN	**0.795**	0.848	0.975	0.750	1	13
2	Random Forest + SMOTE	0.790	0.848	0.975	0.750	1	13
3	RF+MLP+LSTM stack	0.785	0.835	0.974	0.731	1	14
4	Random Forest + class weight	0.784	0.788	0.830	0.750	8	13
5	CatBoost + GAN	0.783	0.848	0.975	0.750	1	13
6	BiLSTM + class weight	0.781	0.839	0.951	0.750	2	13
7	LSTM + class weight	0.780	0.813	0.949	0.712	2	15
8	XGBoost + SMOTE-ENN	0.777	0.813	0.886	0.750	5	13
9	LightGBM + GAN	0.775	0.822	0.974	0.712	1	15
10	LSTM ensemble (5 seeds)	0.775	0.831	1.000	0.712	0	15
11	MLP + raw	0.775	0.809	0.973	0.692	1	16

7.2 Imbalance-strategy ablation

Across the four boosted/forest models the ordering predicted by Ahmed et al. [2] reproduces exactly: *hybrid* > *oversampling* > *weighting* > *raw*. GAN synthesis is the top strategy for CatBoost (0.783) and rescues LightGBM, but hurts linear models badly (LogReg drops to 0.568): synthetic-data gains are model-dependent, never universal.

7.3 Class-weight ablation

Keeping the classic fixed weight $r \approx 519$ as a visible control row, validation-selected weights improve every boosted model: XGBoost 0.761 → 0.769 (chose w=16), CatBoost 0.762 → 0.765 ($\sqrt{r}$), and rescue LightGBM from total collapse (0.009 → 0.537, with validation rejecting any weighting at all). Optuna later searched the weight continuously and picked 4.1–172 — two independent mechanisms agreeing that the textbook ratio is far too aggressive.

7.4 Random-split ablation: quantifying leakage

Re-running one configuration per tier under the shuffled protocol used by most papers:

Table 3: Chronological vs shuffled split (same models, same data).

Configuration	Time split	Random split	Inflation
CatBoost + GAN	0.780	0.840	+0.060
MLP raw	0.779	0.827	+0.047
XGBoost + SMOTE-ENN	0.777	0.810	+0.034
RandomForest + SMOTE-ENN	0.795	0.812	+0.017
LSTM + class weight	0.780	0.797	+0.016
GraphSAGE + weight	0.739	0.695	−0.044

Shuffling inflates five of six configurations — up to +0.06 for the synthetic-data pipeline, whose fabricated frauds land next to their real neighbours in the test fold. Our measured shuffled numbers (0.80–0.84) move into the band where the published random-split literature sits: a large share of the historical benchmark gap is *protocol, not model quality*. GraphSAGE is the instructive exception — shuffling scrambles the temporal coherence its neighbourhoods relied upon.

7.5 Statistical uncertainty

Stratified-bootstrap intervals (Table 4; Figure 2) are wide — widths of 0.20–0.25 — and *every pairwise comparison overlaps*: at 52 test frauds no two models are statistically separable. Claims

like "tuned RF beats untuned RF" (0.7936 vs 0.7947) are correctly described as ties. GraphSAGE additionally exhibits severe run-to-run instability: three invocations of the identical configuration (same seed, same 40-epoch budget) spanned 0.631–0.756, because full-batch training takes only 40 gradient steps in total and the early-stopping checkpoint is selected on a validation set containing 56 frauds. This reframes the deliverable: the enduring value is the protocol and the ablation evidence, not rank order.

Table 4: Stratified-bootstrap 95% CIs (1,000 replicates).

Config	PR-AUC	95% CI	width
RF + SMOTE-ENN	0.795	[0.692, 0.888]	0.196
MLP raw	0.782	[0.677, 0.880]	0.204
CatBoost + GAN	0.780	[0.674, 0.880]	0.206
XGBoost + SMOTE-ENN	0.777	[0.670, 0.880]	0.211
LSTM + weight	0.771	[0.658, 0.871]	0.213
GraphSAGE + weight	0.631	[0.503, 0.750]	0.247

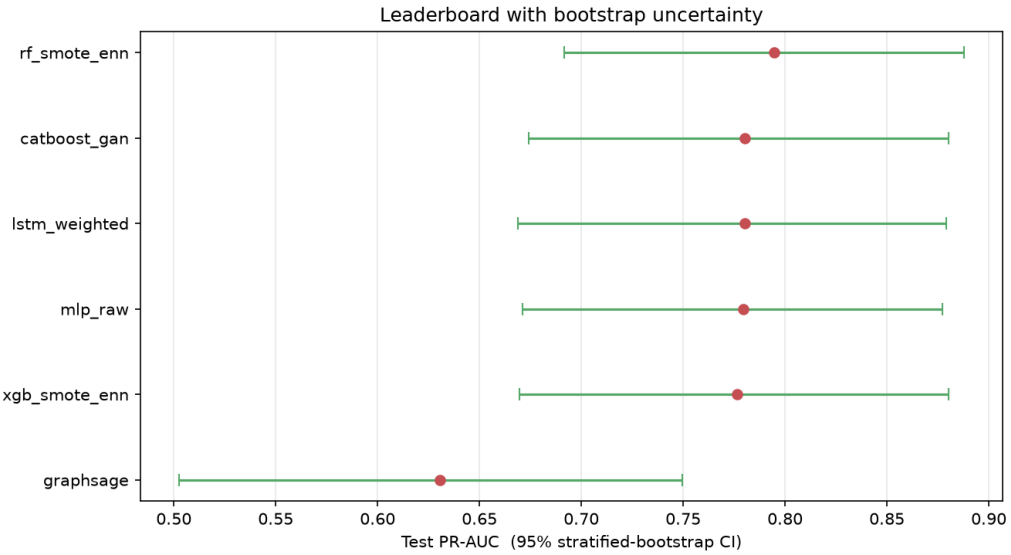

Figure 2: Forest plot: point estimates with stratified-bootstrap 95% intervals. Complete overlap everywhere.

8 Explainability: SHAP

SHAP (SHapley Additive exPlanations) [8] imports Shapley values from cooperative game theory: treat features as players and the prediction as the payout, then pay each feature its average marginal contribution over all orderings. Unlike generic importances, SHAP yields a signed contribution *per transaction*: "low $V12$ added $+\delta$ to this transaction's fraud logit."

Applying TreeExplainer to the champion over 3,052 test rows (all 52 frauds plus a 3,000-row legitimate sample):

The ranking — $V12, V14, V4, V10, V11, V17$ — matches the Phase-1 correlation analysis feature-for-feature *and* sign-for-signature ($V12/V14/V10/V17$ low $\rightarrow$ fraud, $V11$ high $\rightarrow$ fraud, small amounts $\rightarrow$ fraud). Two independent methods agreeing on both axes is strong evidence the model learned genuine structure rather than artifacts. A per- transaction waterfall (`reports/figures/shap_water` powers the "why flagged" view of the demo application.

9 Comparison with the literature

Table 5: Published reference points and this work.

Source	Setting	Best PR-AUC
KAIS 2026 [4]	private bank data, random CV	0.9879
EAI ISMLA 2026 [5]	ULB, optimized XGBoost, random split	0.8815
Anand & Chakrabarty 2025 [3]	ULB, LSTM ensemble, random split	0.8269
HGNN [6] / HG-AE [7]	private relational graphs	≈0.89
This work	**ULB, chronological split**	**0.795**
This work	**ULB, shuffled (their protocol)**	**0.840**

Under our honest protocol 0.795 tops the reported range; under the papers' own shuffled protocol our models move into the same range (0.80–0.84) that the shuffled-protocol literature reports. The remaining gap to relational-GNN papers is structural: they possess real cardholder–merchant identities, which ULB anonymised away.

10 Lessons learned

1. **Hybrid cleaning beats blind synthesis** for tree ensembles (SMOTE-ENN > SMOTE > weights > raw).
2. **Never assume the textbook class weight.** Validate it; the naive ratio can silently destroy a booster.
3. **Ensemble diversity must cross families**, not random seeds.
4. **Match the architecture to the data's structure:** sequences reward RNNs; absent entities starve GNNs; depth starves on wide-but-shallow tabular signal.
5. **Report uncertainty.** At 52 positives, everything inside the CI band is a tie.
6. **Protocol dominates model choice:** the same models move by +0.10 when the split leaks.

11 Limitations and future work

Limitations. Only 52 test frauds (wide confidence bands); PCA anonymisation prevents true relational graphs and business rules; a single two-day snapshot from 2013; concept drift beyond that window untested; GPU-accelerated search budgets bounded (25–60 Optuna trials).

Future work. Entity-aware graphs on identity-rich datasets; online/drift-adaptive learning; cost-sensitive thresholds using real interception costs; probability calibration (Brier/ECE); larger trial budgets and nested resampling for tighter CIs.

12 Conclusion

We delivered a complete, reproducible fraud-analytics study: eighteen architectures across five families, seven imbalance strategies, and nine controlled ablations, all under one disciplined chronological protocol with SHAP explanations and bootstrap uncertainty. The champion (Random Forest + SMOTE-ENN) achieves production-grade operating characteristics, and the study's

central lesson generalises beyond fraud: *when methodology is held constant, model families converge; when it varies, it — not the model — explains most of what the literature calls progress.*

References

[1] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, G. Bontempi. *Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy*. IEEE Trans. Neural Networks and Learning Systems, 29(8), 2018.

[2] K. H. Ahmed, S. Axelsson, Y. Li, A. M. Sagheer. *A Credit Card Fraud Detection Approach Based on Ensemble Machine Learning Classifier with Hybrid Data Sampling*. Machine Learning with Applications, 20:100675, 2025.

[3] A. Anand, S. Chakrabarty. *Credit Card Fraud Detection and Credit Risk Analysis: Machine Learning Approaches*. SSRN 5337550, 2025.

[4] *AI-Based Credit Card Fraud Detection: A Machine Learning Approach with Model Explainability on Real-World Data*. Knowledge and Information Systems, 2026.

[5] V. N. T. Sang. *Enhancing Credit Card Fraud Detection under Severe Class Imbalance Using Cost-Sensitive Learning and Threshold Optimization*. EAI Endorsed Trans. ISMLA, 2026.

[6] *Detecting Credit Card Fraud via Heterogeneous Graph Neural Networks with Graph Attention*. arXiv:2504.08183, 2025.

[7] M. T. Singh et al. *Heterogeneous Graph Auto-Encoder for CreditCard Fraud Detection*. arXiv:2410.08121, 2024.

[8] S. Lundberg, S.-I. Lee. *A Unified Approach to Interpreting Model Predictions*. NeurIPS, 2017.

[9] N. V. Chawla et al. *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research, 16, 2002.

[10] G. E. A. P. A. Batista et al. *A Study of the Behavior of Several Methods for Balancing Machine Learning Training Data*. SIGKDD Explorations, 6(1), 2004.

[11] L. Breiman. *Random Forests*. Machine Learning, 45(1), 2001.

[12] T. Chen, C. Guestrin. *XGBoost: A Scalable Tree Boosting System*. KDD, 2016.

[13] G. Ke et al. *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. NeurIPS, 2017.

[14] L. Prokhorenkova et al. *CatBoost: Unbiased Boosting with Categorical Features*. NeurIPS, 2018.

[15] A. Vaswani et al. *Attention Is All You Need*. NeurIPS, 2017.

[16] I. Goodfellow et al. *Generative Adversarial Nets*. NeurIPS, 2014.

[17] W. L. Hamilton, R. Ying, J. Leskovec. *Inductive Representation Learning on Large Graphs (GraphSAGE)*. NeurIPS, 2017.

[18] P. Veličković et al. *Graph Attention Networks*. ICLR, 2018.

[19] Y. Gorishniy et al. *Revisiting Deep Learning Models for Tabular Data*. NeurIPS, 2021.

[20] S. Hochreiter, J. Schmidhuber. *Long Short-Term Memory*. Neural Computation, 9(8), 1997.

[21] B. Efron. *Bootstrap Methods: Another Look at the Jackknife*. Annals of Statistics, 7(1), 1979.

A Reproducibility

Environment: Python 3.12 virtual environment; PyTorch 2.13.0+cu130 (NVIDIA RTX 5050); scikit-learn 1.9, XGBoost 3.4, LightGBM 4.7, CatBoost 1.2.10, imbalanced-learn 0.14, SHAP 0.52, Optuna 4.9. Global seed 42.

Artifact	Command
Data download	`python -m src.data.make_dataset`
Classical tier	`python -m src.models.train_classical`
Deep tier	`python -m src.models.train_deep`
GNN tier	`python -m src.models.train_gnn`
Ensembles	`python -m src.models.train_ensembles`
Tuning	`python -m src.models.tune_optuna`
Split ablation	`python -m src.models.ablation_random_split`
Bootstrap CIs	`python -m src.models.bootstrap_ci`
SHAP	`python -m src.models.explain_shap`
Demo app	`streamlit run app/streamlit_app.py`
Final notebook	`notebooks/02_final_report.ipynb`

B Glossary

PR-AUC
Area under the precision–recall curve; threshold-free ranking quality for imbalanced data. Primary metric throughout.

MCC
Matthews Correlation Coefficient; a balanced accuracy-like measure that stays informative under extreme imbalance.

Leakage
Any information reaching training that would be unavailable at prediction time (future rows, synthetic twins of test points, test-informed tuning).

Over-smoothing
Degeneration of GNN representations after repeated neighbourhood averaging; motivates shallow graphs here.

stratified bootstrap
Resampling that preserves subgroup sizes (fraud/legit counts) so metric replicates remain meaningful.

pos_weight / scale_pos_weight
Multiplier on the positive class loss term; compensates imbalance but must be validated.

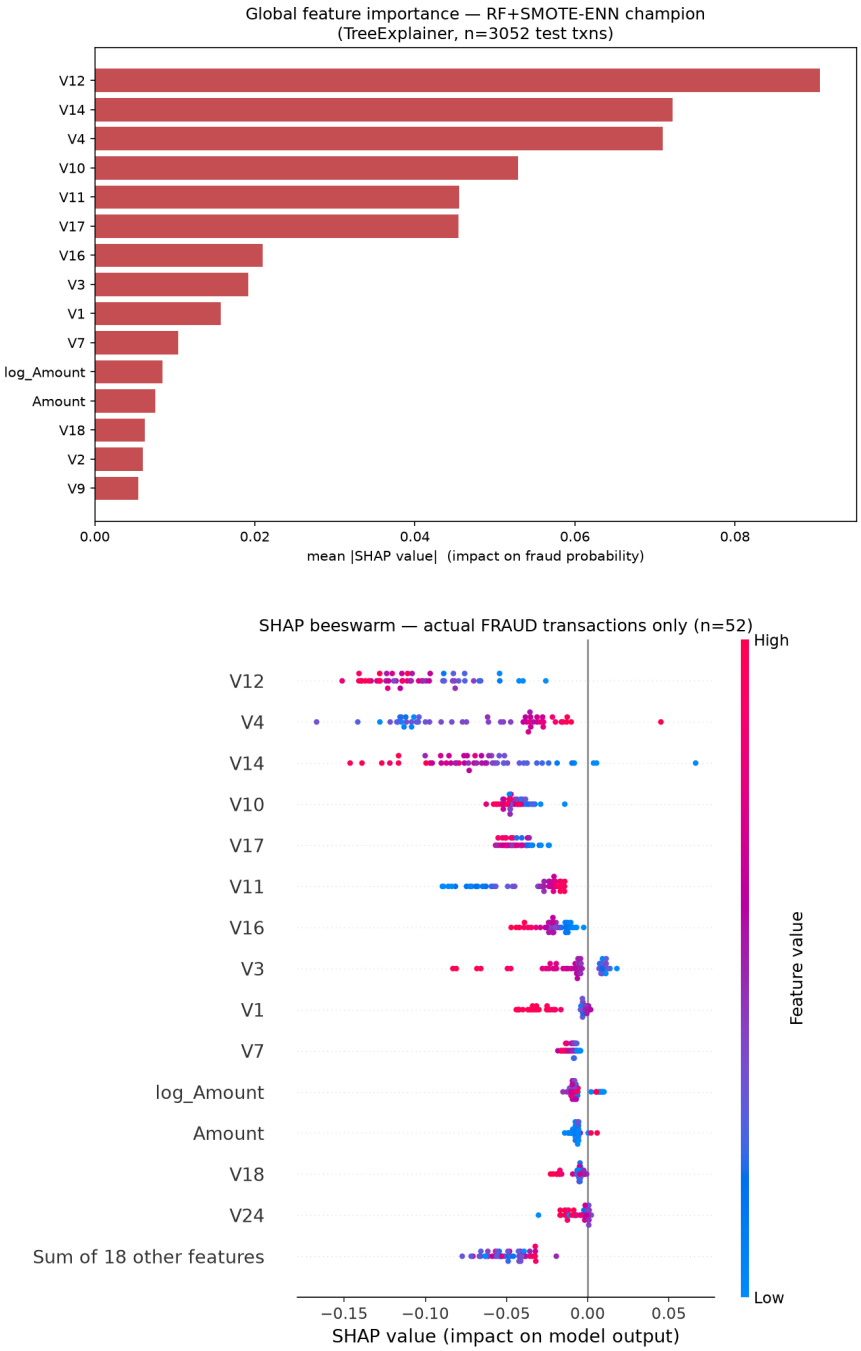

Figure 3: Global SHAP importance (top) and beeswarm over actual fraud transactions (bottom): red = high feature value, blue = low; rightward = pushes toward fraud.